

BUGFIX

A Simple Game Design Review

How I used difficulty, feedback and progression in my first complete game



David Andrei Decă

July 2026

[Play BugFix on itch.io](#) | [LinkedIn](#)

This document is a short and simple look at the design choices behind BugFix. It focuses on what the player sees and feels while playing.

1. About the Project

BugFix is a 2D arcade game made in Godot with GDScript, along with Krita for art. The setting takes place inside the world of an IDE, in which the player moves between code lines and fixes bugs before too many of them end up active at the same time.

Initially, I chose this idea because I wanted to mix programming with game development, but eventually it turned into an actual game that determined me to learn user experience, game design and player satisfaction. Programming errors are usually shown as text, but in BugFix, they become objects that the player can find and repair.

The game includes:

- player movement and a camera that follows the player;
- code lines that are created while the player moves forward;
- eight enemy types with different rules;
- score, combo and high score systems;
- achievements and saved progress;
- music, sound effects and visual feedback.

2. The Main Gameplay Loop

The main idea is easy to understand. The player repeats the same basic steps during the whole run:

Step	Player action
1	Find a code line with a bug.
2	Move close to that line.
3	Read the prompt or understand the enemy rule.
4	Press the correct key or complete the small task.
5	Receive score and continue moving forward.

This loop stays the same, but the way a bug is fixed changes. This helped me add new ideas without changing the whole game every time, basically keeping it simple.

3. Difficulty Progression

At the start, the player only meets the easiest enemy. New enemy types appear later. This gives the player time to learn the controls before the game asks for more attention.

Enemy	Unlock point	What the player must do
Glitchling	Row 0	Press E.
Jester	Row 75	Press a different key shown on screen.
Mirror	Row 150	Read reversed text.
Clicker	Row 300	Press several times quickly.
Counter	Row 300	Enter the correct number.
Mimic	Row 450	Notice that the bug is a trap.
Prisoner	Row 600	Enter a password.
Programmer	Row 1200	Complete several repair steps.

The difficulty does not increase only because the game becomes faster, it also grows because the player must remember more rules and apply them within time. A new player may need more time to read each prompt, whereas a more experienced player can react faster and can handle several bug types at once. This is where skill increase collides with difficulty increase.

My goal was to make the challenge grow together with the player. First, the player learns the basic action, afterward, the same action becomes harder because there are more choices and more pressure.

A possible difficulty problem

Clicker and Counter unlock at the same point. This may result in a bigger jump than the earlier unlocks, and although it can be fine, I decided to test it with more players, since If many players fail here, I would consider moving one enemy to a later row.

The result additionally concluded in a conflict, as player satisfaction may come from the instant reward received by fixing the bugs or advancing fast in skill, contradicting my idea of lowering the difficulty.

Other ways the game becomes harder

- bugs can appear more often in later parts of the game;
- different enemy types can appear in the same time span;
- the player loses when too many bugs are active;
- some bugs take more time or more steps to fix.

4. Feedback and Player Motivation

A player should understand when an action works. For this reason, BugFix uses several types of feedback.

System	What the player sees or hears	Why it is useful
Visual feedback	Red bug lines, green repair flashes, prompts and zone messages.	Shows what needs attention and confirms a repair.
Sound feedback	Sounds for spawns, repairs and menu actions.	Makes actions feel clearer and more satisfying.
Score	Points after a successful repair.	Shows progress during the run.
Combo	A higher combo after fast repairs.	Encourages quick and clean play.
High score	The best result is saved.	Gives the player a reason to try again.
Achievements	Extra goals for fixes, zones and enemy types.	Rewards different ways of playing.

These systems create a simple cycle. The player fixes a bug, receives a clear reward and wants to continue. At the same time, active bugs create pressure, and if the player waits too long, the run becomes harder.

The help tool

The game also has a help tool that can fix one bug automatically. I added it as a recovery option, it can save the player when the screen becomes too busy. It is useful, but it should not appear so often that it removes the challenge.

5. Balance Choices

The main balance problem was simple: if bugs appear too quickly, the game becomes frustrating. If they appear too slowly, the player becomes bored.

I used three main settings to control the difficulty:

1. spawn time - how often a new bug appears;
2. enemy weights - how often each enemy type is chosen;
3. active bug limit - how many bugs can stay on screen before game over.

These settings can be changed without rebuilding the whole game.

6. Testing and Future Improvements

I tested the main functions of the game. I checked movement, spawning, every enemy type, prompts, score saving, menus and game over. This showed that the game works as planned.

However, functional testing is not the same as difficulty testing. A system can work correctly and still feel too easy or too hard. The next step would be to ask more people to play without help from me and to play it myself.

For each test, I would record simple information:

- the row where the player loses;
- which enemy causes the most mistakes;
- how long a normal repair takes;
- how often the help tool is used;
- which part feels too slow, too hard or unclear.

This data would help me make better changes. For example, if most players lose soon after row 300, I would look at the Clicker and Counter unlocks. If players rarely use the help tool, I would check if it is hard to notice or not useful enough.

7. What I Learned

BugFix taught me that making a complete game is not only about writing code. The player must understand the rules, see the result of each action and feel that the difficulty is fair.

The project also taught me to separate systems. Movement, code lines, bug spawning, achievements and menus use different scripts. This made the project easier to fix and improve.

The strongest part of the design is that new enemies change a familiar task. The player always searches for a bug and repairs it, but the exact solution changes. This keeps the game easy to understand while still adding variety.

Project Links

Game: splashthereal.itch.io/bugfix

LinkedIn: linkedin.com/in/davidandreideca

Email: deca.david.andrei@gmail.com